

3rd Lab with VTK (Visualization Tool Kit)

João Varela, 113780

Carolina Prata, 114246

Information Visualization, 2025

MSc in Computer Engineering, University of Aveiro

Abstract—This document provides a brief description of the work developed for the third lab of the Information Visualization course, with the objective of understanding how glyphing, picking, unstructured grids, and the association of scalars with vectors and grids work.

I. INTRODUCTION

The Visualization Toolkit (VTK) is an open-source software framework extensively used in 3D computer graphics, image processing, and scientific visualization, offering robust tools for handling a wide range of data types and supporting visualization methods such as volume rendering, contouring, and streamline analysis. In addition to features such as multiple actors and renderers, shading options, textures, transformations, and observers with callbacks, VTK also provides mechanisms for glyphing, picking, unstructured grid representations, and the association of scalar data with vectors and grids.

II. DEVELOPMENT

The laboratory assignment provided by the course instructor comprises several exercises covering the following topics:

- Glyphing;
- Picking;
- Unstructured Grid;
- Scalar association to vectors and grids;
- HedgeHog;

A. Glyphing

For the first exercise, we explored the `vtkGlyph3D` class to understand the concept of glyphing and how data can be represented using geometric symbols. The objective was to visualize a sphere with cones placed at each vertex of the sphere's polygonal mesh, using the cones as glyphs oriented along the surface normals.

To achieve this, a sphere was generated using `vtkSphereSource` with theta and phi resolutions of 30, providing the input dataset of points where glyphs would be placed on file `glyph.py`. A cone was created using `vtkConeSource` to serve as the glyph source the geometric symbol to be replicated at each sphere vertex. The glyphing process was implemented by creating a `vtkGlyph3D` object and establishing the appropriate connections:

```
glyph = vtkGlyph3D()
glyph.SetSourceConnection(coneSource.GetOutput()
    → tPort())
glyph.SetInputConnection(sphereSource.GetOutput()
    → utPort())
glyph.SetScaleFactor(0.25)
```

```
glyph.SetVectorModeToUseNormal()
```

The `SetScaleFactor` method scales the cones to 25% of their original size, ensuring appropriate proportions relative to the sphere's surface. The `SetVectorModeToUseNormal` method orients each glyph according to the normal vectors at the sphere's vertices, causing all cones to point radially outward, perpendicular to the surface.

Experimenting with different theta and phi resolution values using `SetThetaResolution` and `SetPhiResolution` reveals significant differences in glyph distribution. Lower resolutions (e.g., 10) produce fewer vertices and thus fewer, more sparsely distributed glyphs, while higher resolutions (e.g., 50) generate a denser mesh with more glyphs, creating a smoother and more detailed representation of the sphere's surface structure.

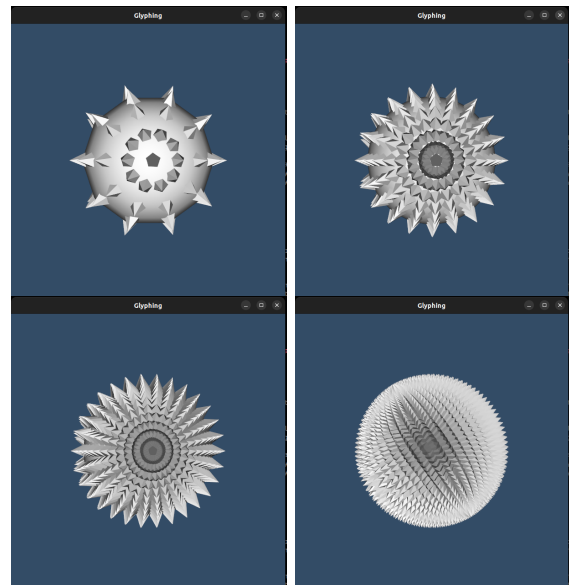


Fig. 1. Results obtained by executing the first exercise with different values for the Scale Factor, Theta, and Phi resolution, using 10, 20, 30, and 50, respectively.

B. Picking

In this exercise, in file `pick.sample`, a point picking mechanism was added to the interactor of the previous example in order to enable the selection of points on the rendered model. This functionality allows the user to interactively identify specific positions on the surface during execution.

To visually represent the selected point, an additional sphere actor was created. Its visibility was initially disabled using

`VisibilityOff()`, ensuring that it would only become visible when a pick event occurs. A callback class was then implemented to handle the picking event and update the scene accordingly:

```
def __init__(self, picker, sphere_actor,
    ↪ render_window):
    self.picker = picker
    self.sphere_actor = sphere_actor
    self.render_window = render_window

def __call__(self, obj, event):
    pickPosition =
    ↪ self.picker.GetPickPosition()
    print(f"Picked point coordinates:
    ↪ {pickPosition}")

    self.sphere_actor.SetPosition(pickPosition)
    ↪ n)
    self.sphere_actor.VisibilityOn()
    self.sphere_actor.GetProperty().SetColor(
    ↪ 1.0, 0.0, 0.0)
    self.render_window.Render()
```

This callback class is responsible for handling the pick event. When a point is selected, the `__call__` method retrieves the coordinates of the picked position using `GetPickPosition()`. The sphere actor is then moved to the selected location, its visibility is enabled, and its color is set to red in order to clearly highlight the picked point. The render window is subsequently updated to reflect these changes. Additionally, the picked coordinates are printed to the terminal for verification purposes.

The picking mechanism was configured by creating an instance of `vtkPointPicker`, referred to as `myPicker`. The callback was instantiated as `mol = vtkMyCallback(myPicker, selectedSphereActor, renderWindow)` and registered with the picker using `AddObserver(vtkCommand.EndPickEvent, mol)`. Finally, the picker was associated with the interactor through `iren.SetPicker(myPicker)`. The picking action is triggered during interaction by pressing the P key.

```
Picked point coordinates: (0.11931833432483673, -2.9004027320036316e-18, -0.9207531809806824)
Picked point coordinates: (0.11931833432483673, -2.9004027320036316e-18, -0.9207531809806824)
Picked point coordinates: (-3.4012584686279297, 1.0180739164352417, -3.0)
Picked point coordinates: (-1.3913553953170776, -1.873215675354004, -1.6527036428451538)
Picked point coordinates: (-2.875230312347412, 0.3551308512687683, -3.6616392135620117)
```

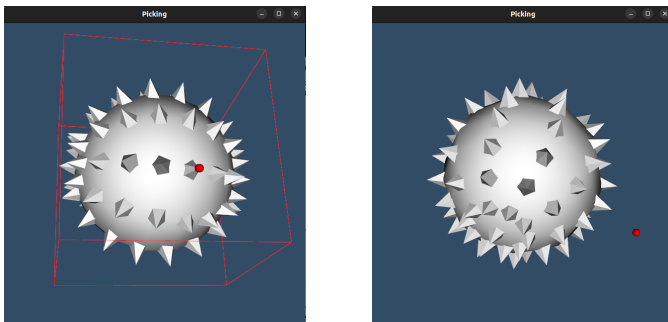


Fig. 2. Result of the execution of the second exercise)

C. Display Coordinates

As an extension to the object picking exercise, an additional task was implemented in which the picked point coordinates are displayed directly within the renderer window, rather than being printed to the terminal. To achieve this, a text visualization mechanism was introduced using a `vtkTextMapper` combined with a 2D actor, enabling the coordinates of the selected point to be rendered on screen, in `coordinates.py`.

Similarly to the previously used sphere actor, the text actor was initially created with its visibility set to off, ensuring that no text is displayed until a point picking event occurs. When such an event is triggered, the callback function is updated to convert the picked position into a formatted string representing the (x, y, z) coordinates with two decimal places. The text mapper is then updated with this information, the text actor is made visible, and its position is adjusted with a small offset relative to the picked point to improve readability. Finally, the render window is updated to reflect these changes.

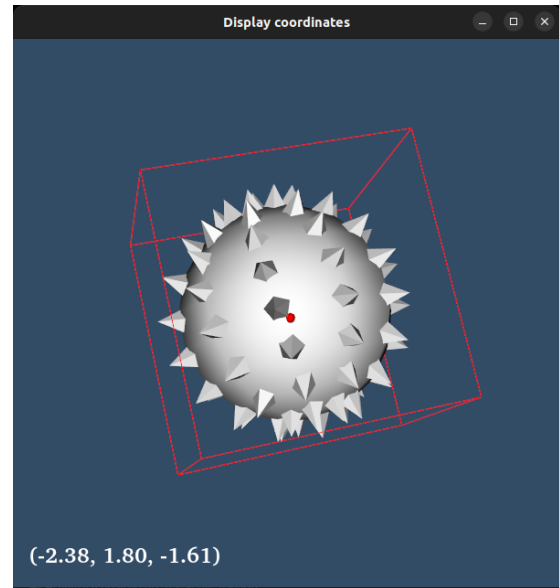


Fig. 3. Point picking visualization displaying selected coordinates in the render window

D. Unstructured Grid

For this exercise, a source file was provided containing code that makes use of VTK's `vtkUnstructuredGrid`. The implementation constructs an unstructured grid composed of a single cell, specifically a tetrahedron. Upon executing the file, the following result was obtained:

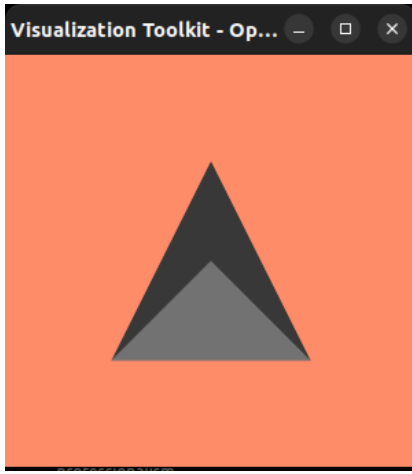


Fig. 4. Result of the execution of the file `ugrid.py` provided by the professor

In this exercise, the initial implementation displaying a tetrahedral cell (`VTK_TETRA`) was modified so that each vertex is represented as an independent cell using `VTK_VERTEX`. Instead of a single tetrahedron composed of four connected vertices, individual vertex cells were created, treating each point as a separate entity within the unstructured grid.

This was achieved by iterating over the list of point coordinates and, for each coordinate, creating a corresponding `VTK_VERTEX` cell. Additionally, the visual properties of the actor were adjusted by setting its color to red and increasing the point size to five, thereby improving the visibility of the individual vertices. The renderer background color was also changed to a light gray in order to enhance contrast.

As a result of these modifications, the visualization displays clearly defined red points, rather than a connected tetrahedral structure.

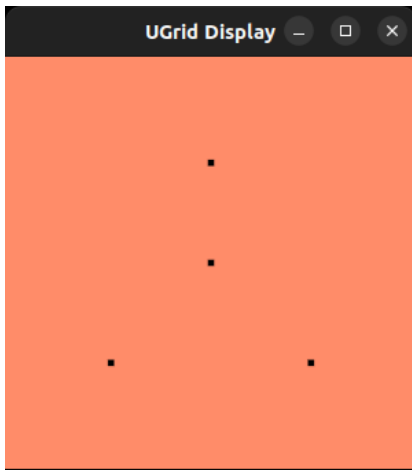


Fig. 5. Results obtained by executing the `ugrid.py` file after changes

E. Scalar association to vectors and grids

In this section, the exercise focused on the creation of an unstructured grid composed of four points, with each point associated with both a vector and a scalar value. The vector

data, defined as $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$, and $(1, 1, 1)$, were stored in a `vtkFloatArray` with three components and used to define the orientation of the glyphs. These vectors were subsequently applied to orient cone glyphs positioned at each grid point using the `vtkGlyph3D` class.

In addition to the vector data, scalar values $(0.1, 0.3, 0.5, 0.8)$ were assigned to the corresponding points and stored in a separate `vtkFloatArray`. These scalar values were employed to control the coloring of the cones, enabling a visual representation of scalar variation across the grid.

The implementation begins by creating an unstructured grid and inserting the four points as individual `VTK_VERTEX` cells. The vector and scalar arrays are then defined and associated with the grid points. A `vtkGlyph3D` object is used to generate cone glyphs at each point, with their orientation driven by the vector data and their appearance influenced by the associated scalar values. The resulting visualization is rendered against a light gray background, as shown below.



Fig. 6. Results obtained by executing the `scalar.py` file

F. HedgeHog

In the final exercise, the visualization of vector data was explored using the `vtkHedgeHog` class. In contrast to the previous exercise, where vectors were represented using cone glyphs, `vtkHedgeHog` depicts vector data as line segments, providing a simpler and more direct representation of vector direction and magnitude.

To accomplish this task, the implementation from the previous exercise was adapted by replacing the `vtkGlyph3D` class with `vtkHedgeHog`. The hedgehog object was configured to use the vector data associated with the unstructured grid and scaled with a factor of 0.3 in order to control the length of the resulting line segments. The outcome of this modification is illustrated by the visualization obtained below.

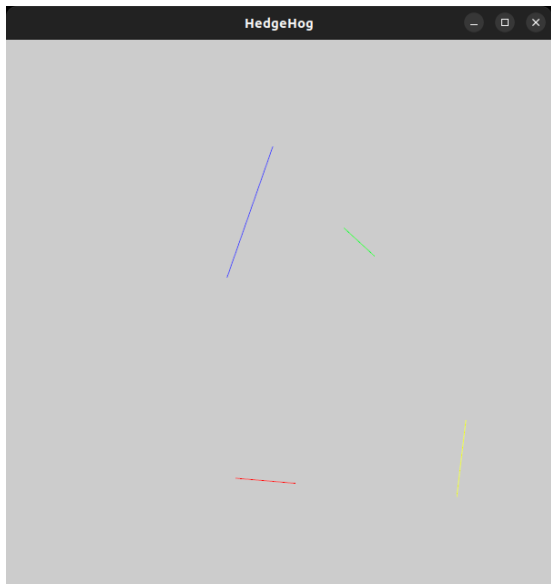


Fig. 7. Results obtained by executing the `hedge.py` file